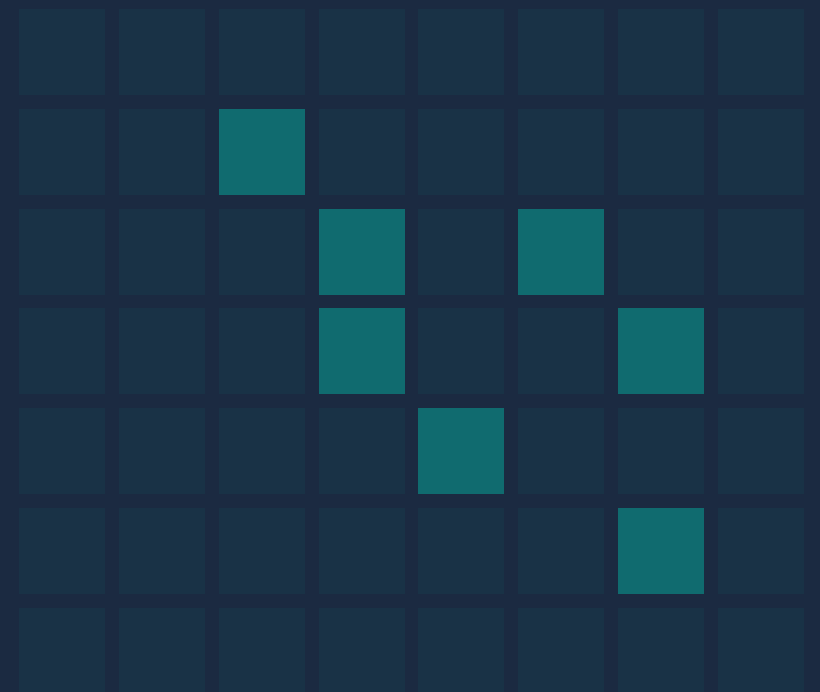


Métodos de Búsqueda

Trabajo Práctico N° 1 — 8-Puzzle y Sokoban

Grupo 8



¿Por qué Sokoban y no Grid World?

01 Un solo agente · BFS, DFS, Greedy y A* con heurísticas admisibles se ven aisladas, sin coordinación multiagente

02 Complejidad controlable · escala con cajas y geometría del tablero (3 niveles), no con el producto de N agentes

03 Demo en vivo · GUI jugable + notación estándar de mapas (XSB)

04 Heurísticas ricas · asignación caja $\rightarrow$ objetivo (algoritmo Húngaro), que en Grid World no aparece igual

Grid World multiagente infla el estado a $(pos_1, \dots, pos_N)$ y mezcla búsqueda con conflictos entre agentes.

Agenda

01

Ejercicio 1

8-Puzzle: estado, paridad y heurísticas

02

Modelado

Cómo representamos Sokoban y su poda

03

Protocolo

Heurísticas y cómo medimos

04

Level 1 y 2

Contraste limpio entre métodos

05

Level 3

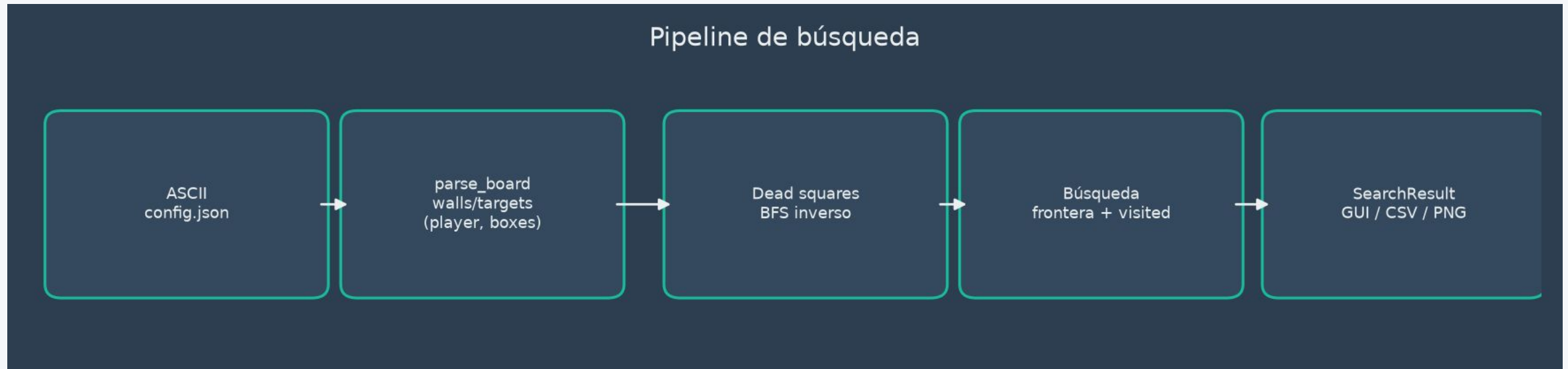
Escala, memoria y trade-offs

06

Conclusiones

Qué gana cada método y cuándo

Pipeline del motor de búsqueda



ASCII (XSB) → parse_board → conjuntos de coordenadas → frontera / visited → SearchResult

Estructura de estado

```
# tablero inicial del enunciado  
(5, 7, 3,  
 8, 2, 0, ← 0 = espacio vacío  
 1, 6, 4)
```

Tupla inmutable de 9 enteros, recorrida por filas

Hasheable el set de visitados consulta en $O(1)$ promedio

Compacta más liviana que una matriz 3×3

- Estado $\neq$ nodo: padre, acción, profundidad y costos g / h / f pertenecen al nodo de búsqueda, no a la identidad del estado
- La posición del vacío puede guardarse en el nodo para no recorrer la tupla en cada expansión

Acciones y las tres metas

Acciones: mover el vacío U / D / L / R · costo 1 · factor de ramificación 2–4

1	2	3
4	5	6
7	8	

G1 · clásica

3	2	1
6	5	4
	8	7

G2 · reflejo

3	6	
2	5	8
1	4	7

G3 · rotación 90°

Rotar o reflejar NO es una acción legal del 8-puzzle

Las tres pertenecen a la misma órbita geométrica D_4 , pero para el algoritmo son tres estados distintos: la única acción permitida es intercambiar el vacío con una ficha adyacente.

¿Por qué este estado tiene 17 inversiones?

En el 8-puzzle, se cuentan pares fuera de orden ignorando el espacio vacío.

5	7	3
8	2	
1	6	4

1 **Secuencia (sin el vacío):**

5, 7, 3, 8, 2, 1, 6, 4

2 **¿Qué es una inversión?**

Una inversión es un par (a, b) con a antes que b y $a > b$.

Ejemplos: $(5,3)$ o $(7,2)$

3 **Conteo de inversiones:** para cada número, contamos cuántos menores hay a su derecha.

Elemento (a)	Números a la derecha	Menores a la derecha	# de inversiones	Pares de inversión formados
5	7, 3, 8, 2, 1, 6, 4	3, 2, 1, 4	4	$(5,3), (5,2), (5,1), (5,4)$
7	3, 8, 2, 1, 6, 4	3, 2, 1, 6, 4	5	$(7,3), (7,2), (7,1), (7,6), (7,4)$
3	8, 2, 1, 6, 4	2, 1	2	$(3,2), (3,1)$
8	2, 1, 6, 4	2, 1, 6, 4	4	$(8,2), (8,1), (8,6), (8,4)$
2	1, 6, 4	1	1	$(2,1)$
1	6, 4	—	0	—
6	4	4	1	$(6,4)$
4	—	—	0	—

4 **Total = 4 + 5 + 2 + 4 + 1 + 0 + 1 + 0 = 17 inversiones**

★ El espacio vacío no se cuenta.

Verificación exhaustiva por fuerza bruta

181.440

Estados visitados desde G1 — exactamente $9! / 2$

14

Distancia mínima de G1 a G3

G2: NO

G2 no aparece — inalcanzable desde G1

- Se corrió un BFS partiendo de G1, sin límite de profundidad
- **La búsqueda recorrió por completo la componente conexa de G1: la ausencia de G2 es una prueba, no una sospecha**

Heurísticas admisibles — de más débil a más informada

1 Fichas fuera de lugar

Estado actual Meta

2	1	3
4	5	6
7	8	

1	2	3
4	5	6
7	8	

! Cuenta cuántas fichas no están en su lugar

$h = 2$

2 Distancia Manhattan

Estado actual Meta

2	1	3
4	5	6
7	8	

1	2	3
4	5	6
7	8	

Suma distancias horizontales y verticales

$1 + 1 = 2$

$h = 2$

3 Manhattan + conflicto lineal

Estado actual Meta

2	1	3
4	5	6
7	8	

1	2	3
4	5	6
7	8	

Hay un conflicto lineal: **+2 extra**

Una ficha debe salir de la fila y volver

$h = 2 + 2 = 4$

RECOMENDADA

Manhattan + conflictos lineales

Suma 2 por cada conflicto independiente.
Admisible y domina a Manhattan.

Como hay varias metas alcanzables: $h(s) = \text{mínimo de } h_G(s) \text{ sobre cada meta alcanzable } G$. El espacio vacío no se incluye en ninguna de las heurísticas.

A* paso a paso en el 8-puzzle

Ejemplo correcto usando $h = \text{Manhattan} + \text{conflictos lineales}$

Meta

1	2	3
4	5	6
7	8	—

1. Estado inicial

1	2	3
4	—	6
7	5	8

$g = 0$

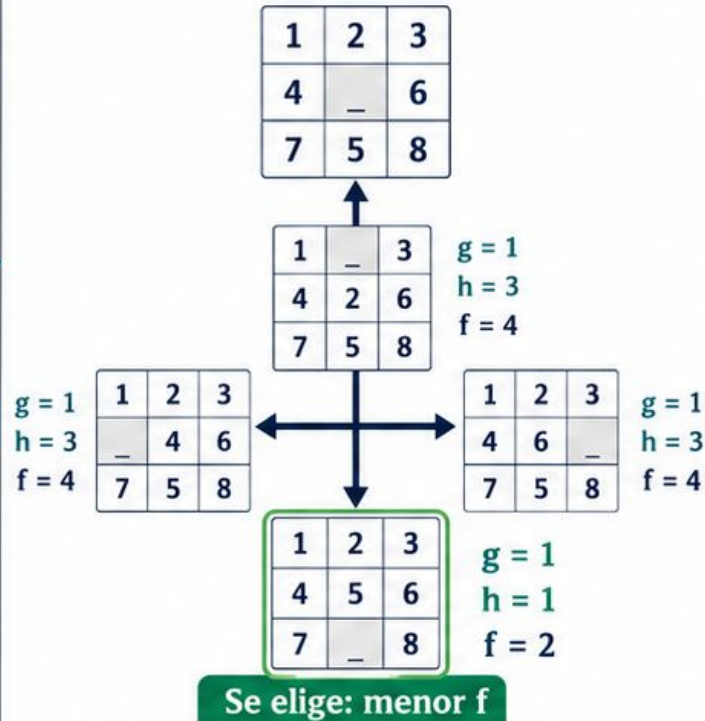
$h = 2$

5 está a 1, 8 está a 1,
sin conflicto lineal

$f = 2$

2. Expandir vecinos

Se extrae el menor f de OPEN



3. Nuevo estado extraído

1	2	3
4	5	6
7	—	8

$g = 1$

$h = 1$

$f = 2$

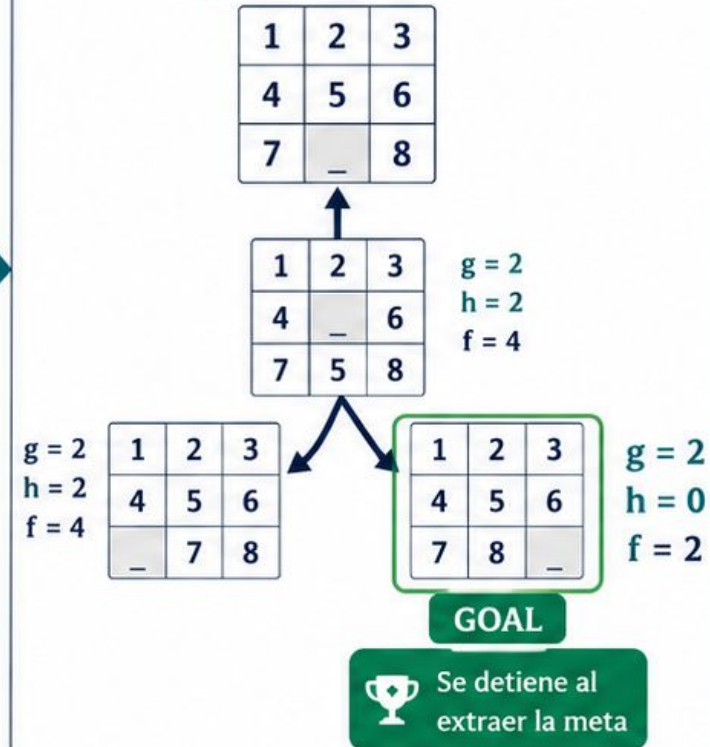
OPEN (frontera)



Mejor para expandir:
 $f = 2$
(estado mostrado)

4. Siguiente expansión

Se expande el estado extraído



$$f(n) = g(n) + h(n)$$

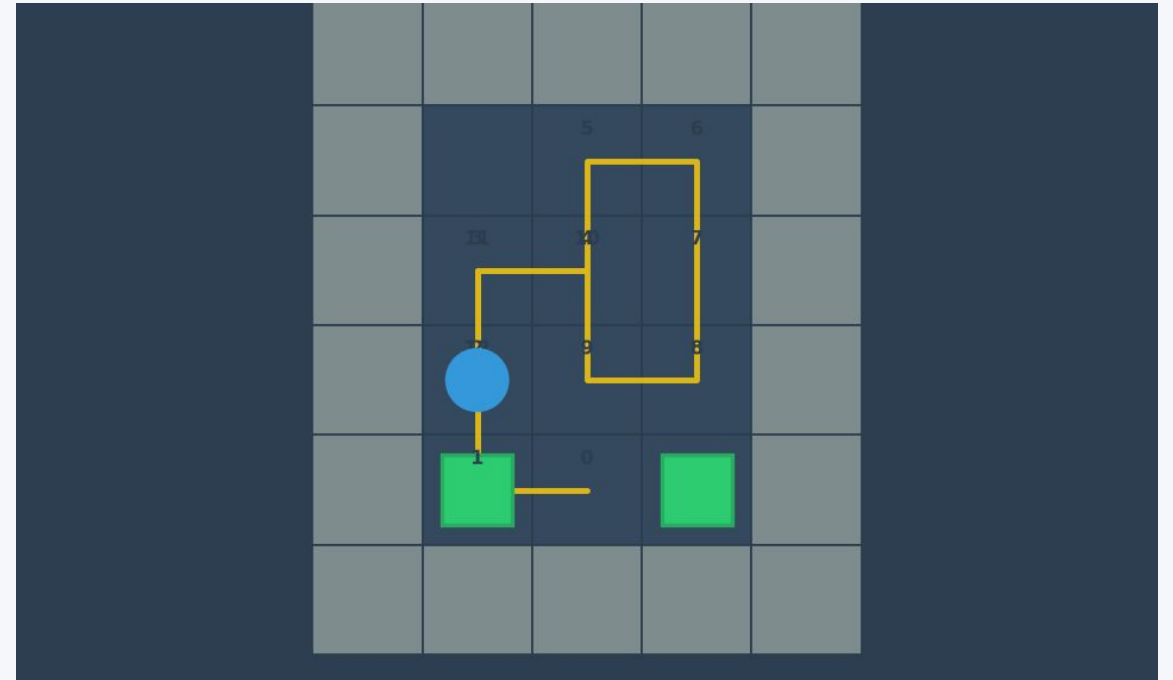
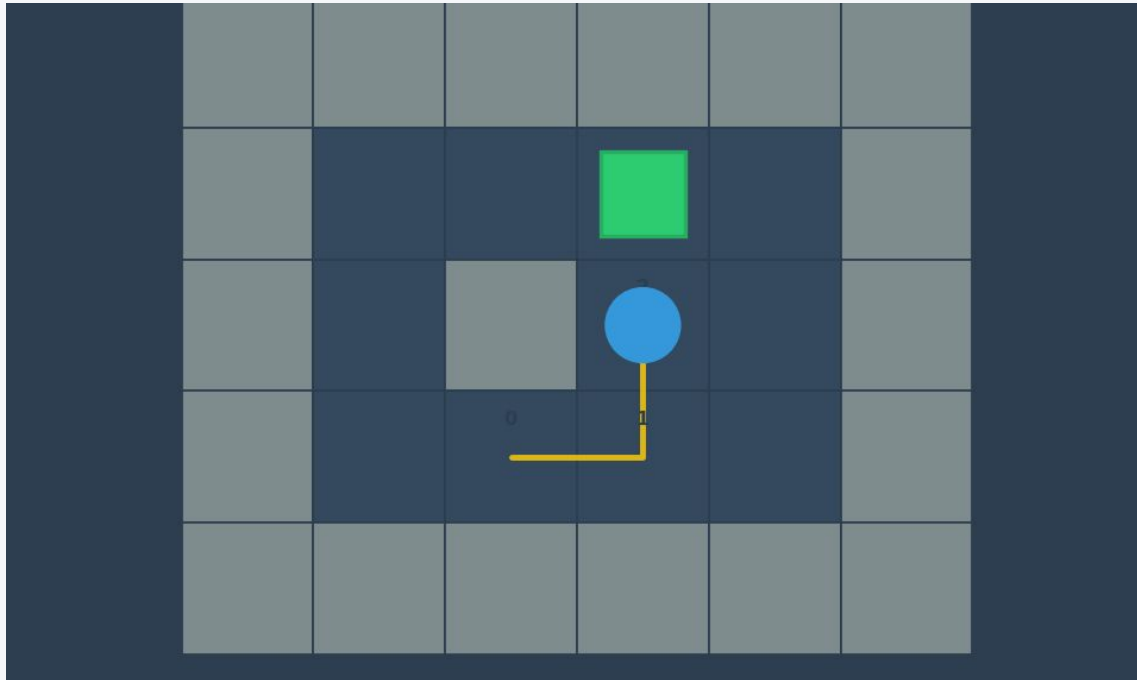


A* prioriza el menor f



Una heurística más informada
= menos expansiones

Así se ve un Sokoban resuelto



Estructura de estado

```
estado = (  
    player: (r, c),  
    boxes: frozenset[(r, c)]  
)  
# walls y targets NO van en el estado
```

Muros y objetivos estáticos: no cambian durante la búsqueda

Acción un paso del jugador

Empujar cuesta igual que moverse costo uniforme

- **Meta:** boxes == targets — todas las cajas sobre una celda objetivo

Optimizamos la cantidad de movimientos del jugador, no solo los empujes: por eso cada paso cuenta igual.

Representación del tablero

CAPA 1 · SERIALIZACIÓN

config.json / LEVELS

list[str] — el mapa en texto, notación XSB.
Legible y editable a mano.

CAPA 2 · GUI EN VIVO

SokobanGame.grid

matriz de caracteres mutable, solo para pintar,
teclas y undo.

CAPA 3 · BÚSQUEDA

parse_board()

conjuntos de coordenadas: walls, targets
(estáticos) + boxes, player (el estado).

Notación XSB: “#” = muro · “.” = objetivo · “\$” = caja · “@” = jugador · “” = caja+objetivo · “+” = jugador+objetivo.*

Glosario

Estado

(player, boxes) — frozenset hasheable

Nodo

estado + path + costo acumulado

Frontera

generados, todavía no expandidos

Visitados

estados que ya no se re-encolan

Expandir

visitar un nodo frontera y agregar sus sucesores al conjunto frontera

$f(n)$

BFS: $f = g$
Greedy: $f = h$
A*: $f = g + h$
DFS: no usa

Poda: Dead Square Pruning

1

Dead Square

búsqueda inversa desde los objetivos hacia cada celda para descartar aquellas que no tienen manera de llevar a una solución

2

Descarte en la generación

es un cálculo previo a la búsqueda (costo fijo) donde se crea una colección de celdas inútiles para poder descartar nodos durante la búsqueda

3

Chequeo local

la función `is_blocked` chequea que no caiga en esquinas durante la búsqueda

LEVEL 3

17 / 40

celdas jugables
son potenciales dead squares

Si una caja no puede llegar a ningún objetivo ni siquiera en un tablero vacío, ningún camino solución pasa por esa casilla. No se pierden soluciones ni se afecta la optimalidad.

Impacto de la poda (Level 3, BFS)

NO CHEQUEA DEAD SQUARES

1.709.332

nodos expandidos

Costo: 277 (óptimo)

Tiempo: ~5.8 s



CHEQUEA DEAD SQUARES

836.594

nodos expandidos

Costo: 277 (óptimo, sin cambios)

Tiempo: ~2.9 s

Costo fijo inicial que se espera compensar con menos expansiones de nodos. Muy útil en niveles complejos, no tanto en niveles triviales.

Algoritmos implementados

NO INFORMADO

BFS

$$f(n) = g(n)$$

Frontera: cola LIFO.

Óptimo con costo uniforme.

NO INFORMADO

DFS

f: no usa

Frontera: pila FIFO

No es óptimo.

INFORMADO

Greedy (GGS)

$$f(n) = h(n)$$

Frontera: cola ordenada según $h(n)$.

No es óptimo

INFORMADO

A*

$$f(n) = g(n) + h(n)$$

Frontera: cola ordenada según $f(n)$.

Óptimo si h es admisible.

*Para simplicidad de código se usó la ecuación $f(n)=w*h(n)+(1-w)*g(n)$, donde w vale 0 en BFS, 1 en GGS y 0.5 en A*.*

Tres heurísticas (Sokoban)

ADMISIBLE

Manhattan

Suma de distancias Manhattan de cada caja al objetivo más cercano.
Puede asignar dos cajas al mismo objetivo → subestima más.

ADMISIBLE

Manhattan+Hungarian

Hungarian tiene en cuenta que cada objetivo lo puede ocupar solo una caja y suma distancias Manhattan buscando la mejor asignación caja-objetivo.

NO ADMISIBLE

Weighted

$3 \times \text{hungarian}$ — es WA^* con $w = 3$.



Admisible significa $h(s) \leq h^(s)$ para todo estado s . simple y hungarian ignoran muros y al jugador, por eso siempre subestiman el costo real.*

Por qué importa la admisibilidad

277

A* con heurística admisible
Camino óptimo

291

Costo A* con weighted en Level 3

- Multiplicar por 3 puede hacer $h(s) > h^*(s)$: en Level 1 ya pasa en el estado inicial ($h = 3$ vs. $h^* = 2$)
- **Consecuencia directa: El método deja de ser óptimo**

Qué medimos

LAS 6 MÉTRICAS QUE PIDE EL ENUNCIADO

Éxito / fracaso

Costo de la solución

Nodos expandidos

Nodos frontera

Camino solución

Tiempo de procesamiento

LO QUE AGREGAMOS POR NUESTRA CUENTA

Frontera máxima alcanzada (no solo la final)

Memoria pico — tracemalloc

Convención fija para todos los algoritmos: se expande DESPUÉS de chequear meta, así que el nodo solución no cuenta como expandido. La misma regla en los cuatro métodos hace justa la comparación.

Protocolo de benchmarks

5 / 3

repeticiones — 5 en Level 1 y 2, 3 en Level 3

180 s

timeout en Level 3 (30 s por defecto)

104

filas en benchmarks.csv — una por corrida

- **Costo y nodos expandidos son determinísticos entre repeticiones; el tiempo varía → se reporta media $\pm$ desvío**
- Todas las corridas del CSV oficial miden memoria de la misma forma, así que los tiempos entre algoritmos sí son comparables entre sí

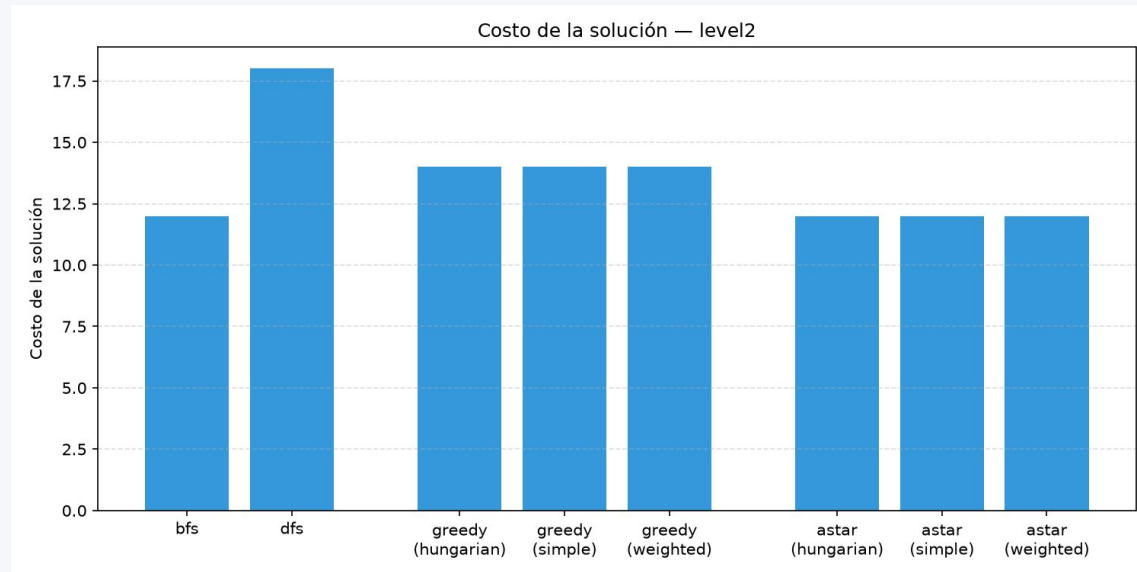
Level 1 — sólo números

Algoritmo	Costo	Expandidos	Tiempo medio
BFS	2	4	~0.10 ms
DFS	2	9	~0.09 ms
Greedy (todas las heur.)	2	3	~0.08–0.15 ms
A* (todas las heur.)	2	3	~0.08–0.16 ms

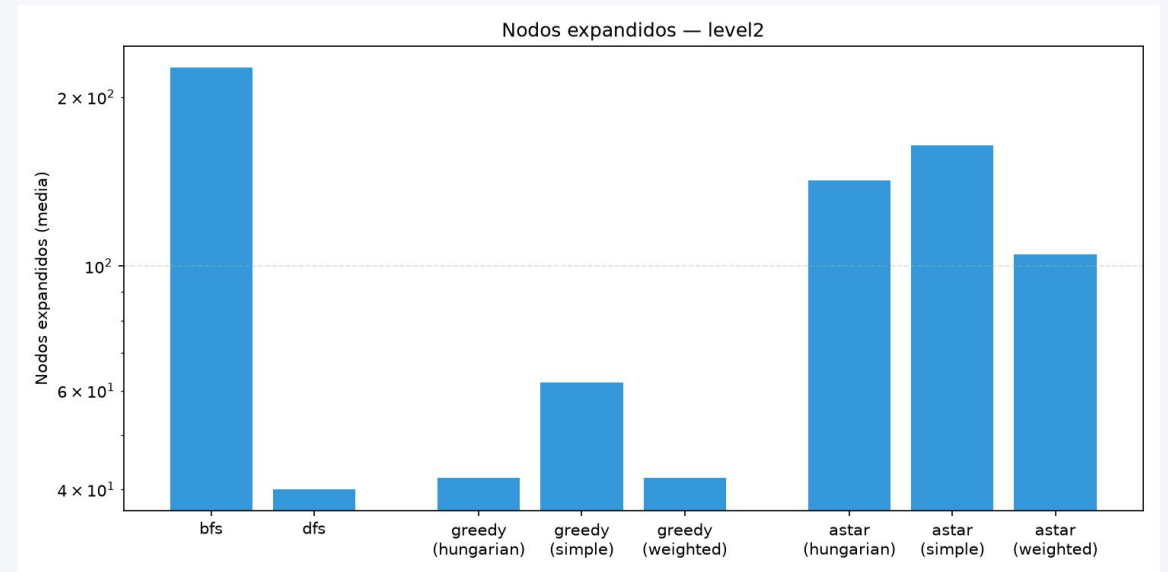
Tablero trivial: todos los métodos, incluso weighted, encuentran el óptimo (2). Con 1 sola caja, simple y hungarian coinciden.

Level 2 — Costo y nodos expandidos

Costo de la solución



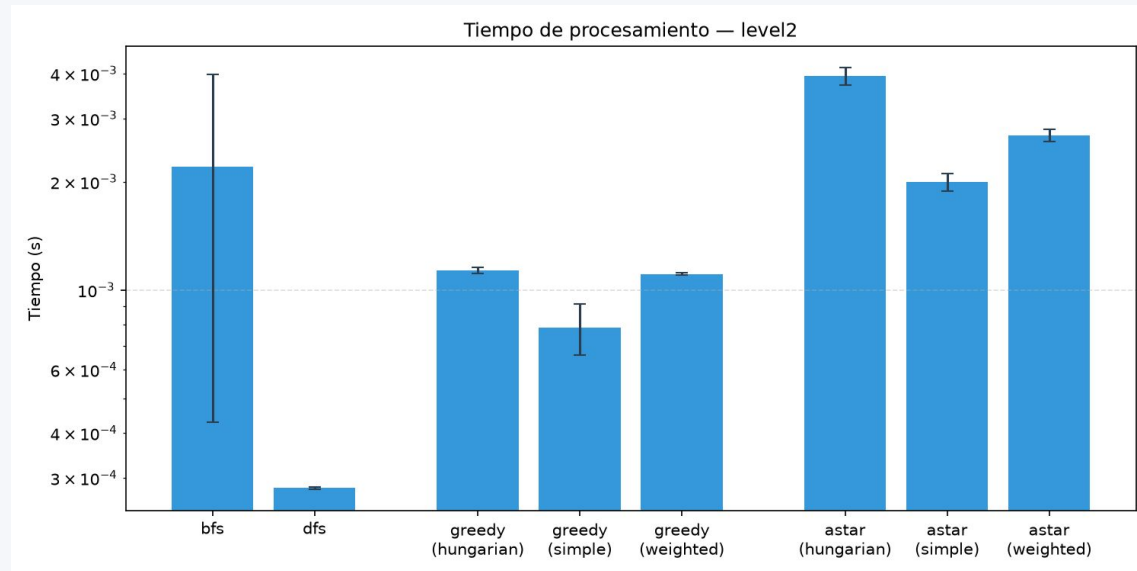
Nodos expandidos (escala log)



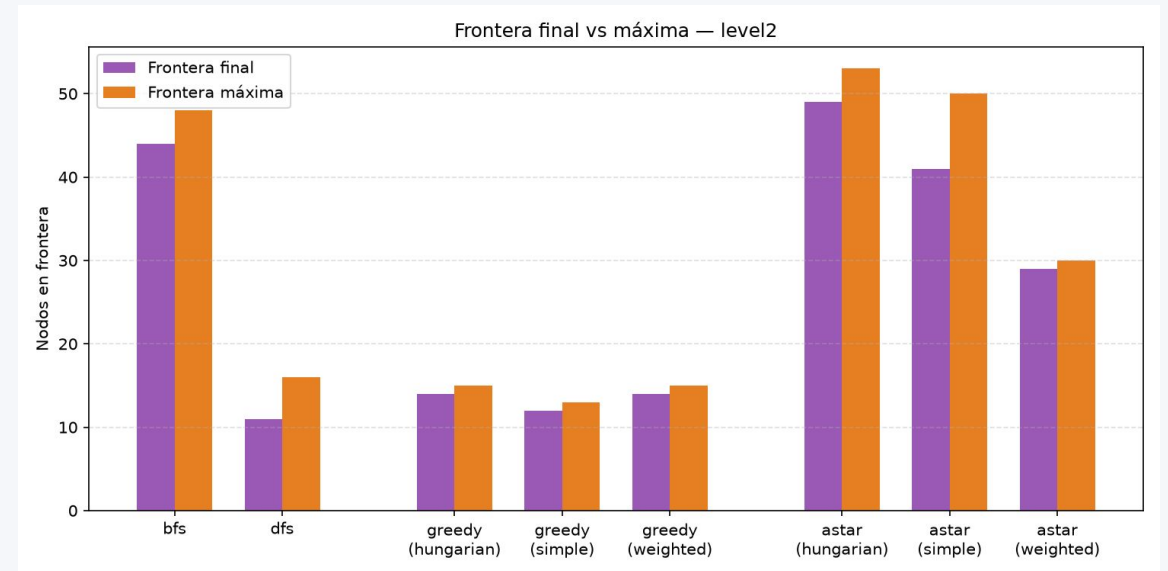
BFS = A = 12 (óptimo) · Greedy = 14 (×1.17) · DFS = 18 · A* hungarian expande 142 nodos vs. 164 de simple y 226 de BFS*

Level 2 — Tiempo y frontera

Tiempo de ejecución (log + error)



Frontera final vs. máxima



DFS gana en tiempo con poquísimos nodos · A hungarian es más lento que simple pese a expandir menos: overhead del solver de asignación*

Level 3 — el mapa grande



Camino óptimo (A* hungarian) — 277 pasos

5

cajas y objetivos

277

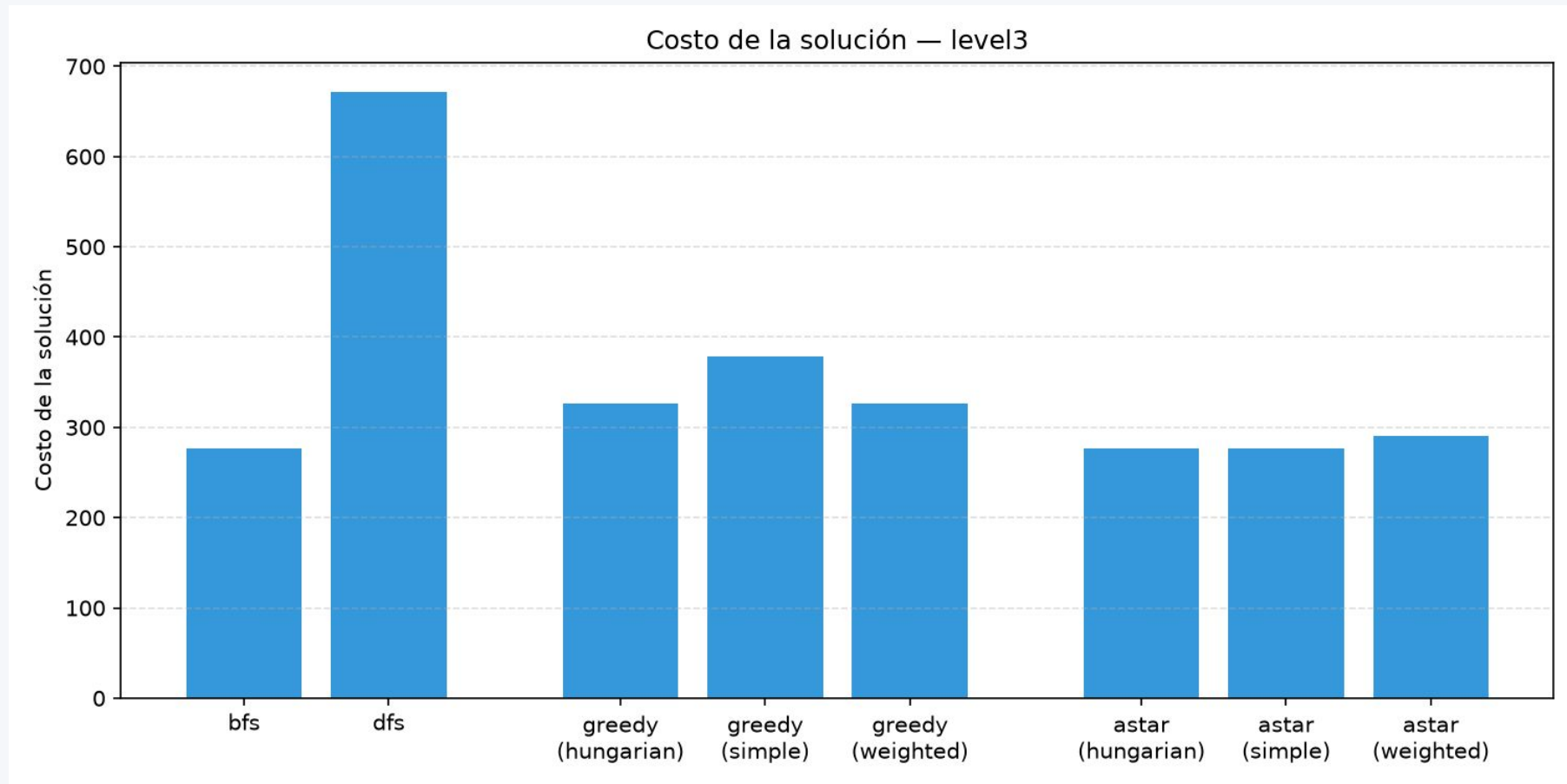
pasos del camino óptimo

17 / 40

celdas son potenciales dead squares

“Acá empieza a doler el espacio de estados.”

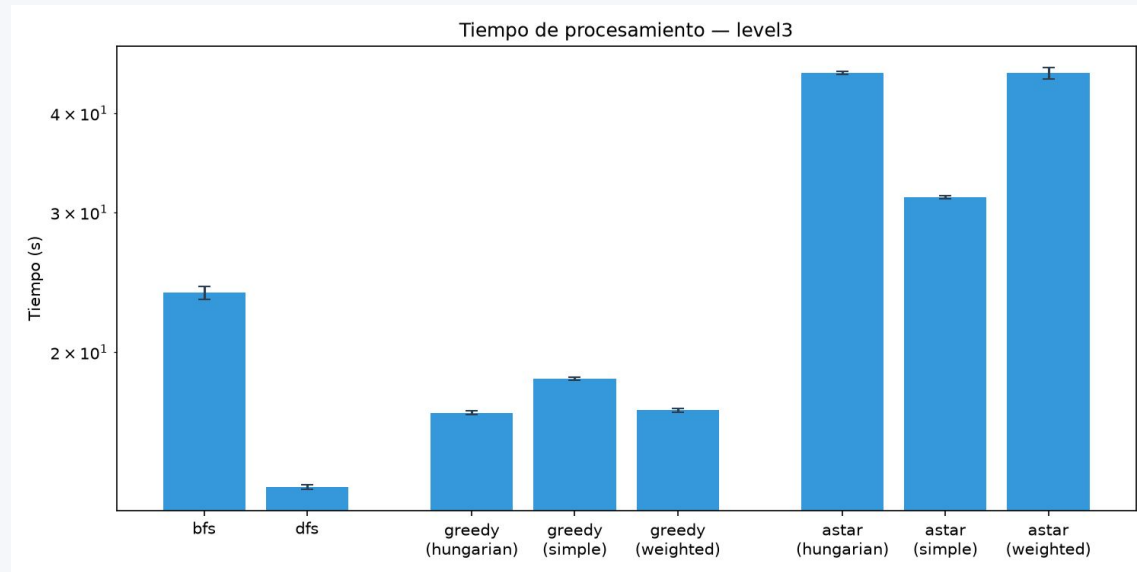
Level 3 — Costo y suboptimalidad



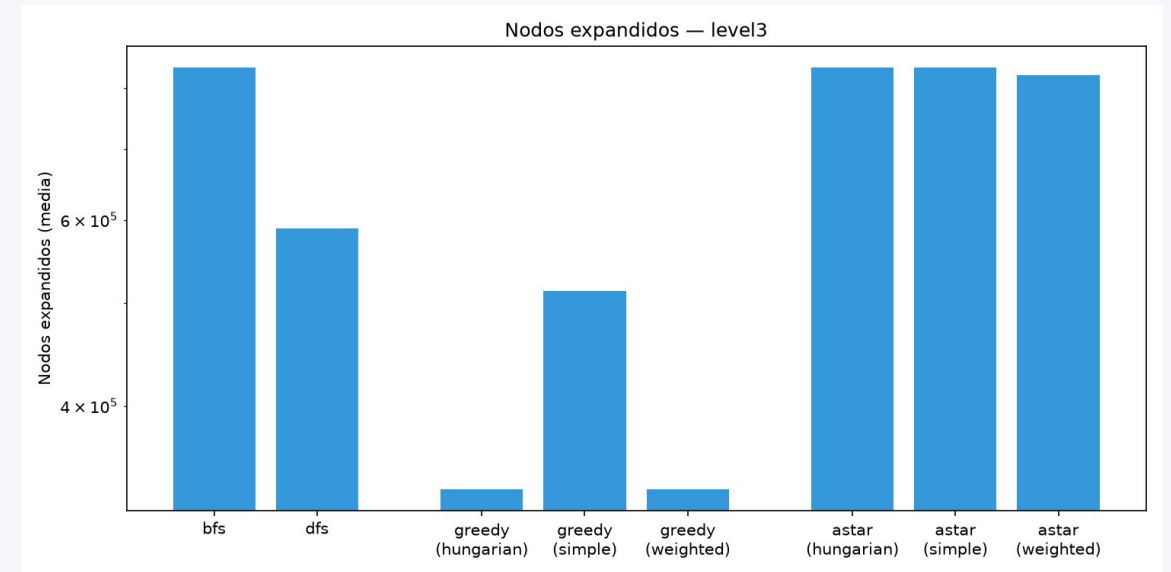
BFS / A admisible = 277 (óptimo) · A* weighted = 291 · Greedy hungarian = 327 (×1.18) · Greedy simple = 379 (×1.37) · DFS = 671 (×2.42)*

Level 3 — Tiempo y nodos expandidos

Tiempo de ejecución (log + error)



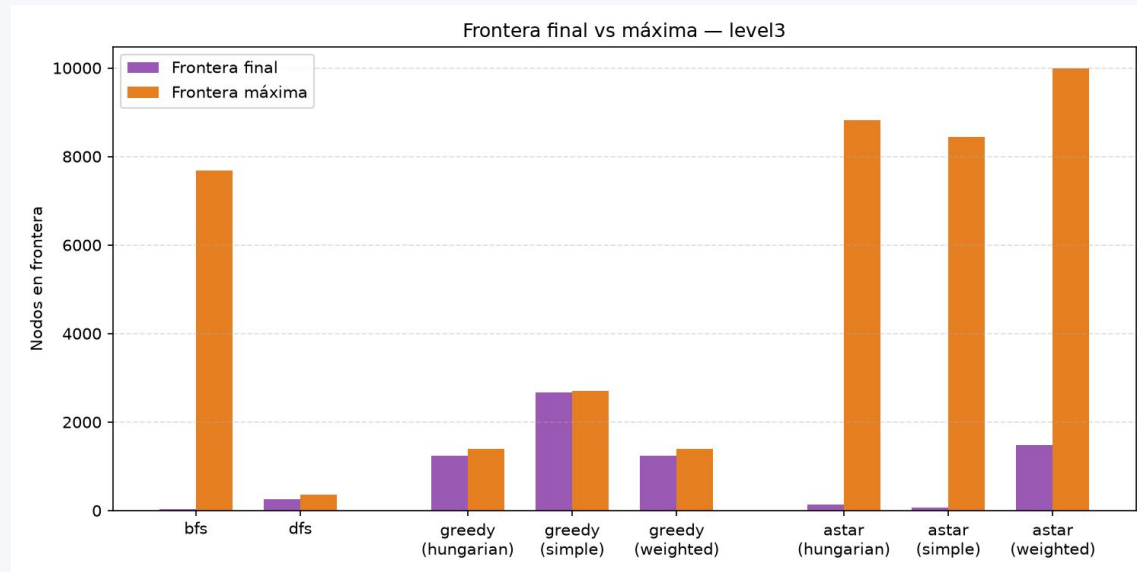
Nodos expandidos (log)



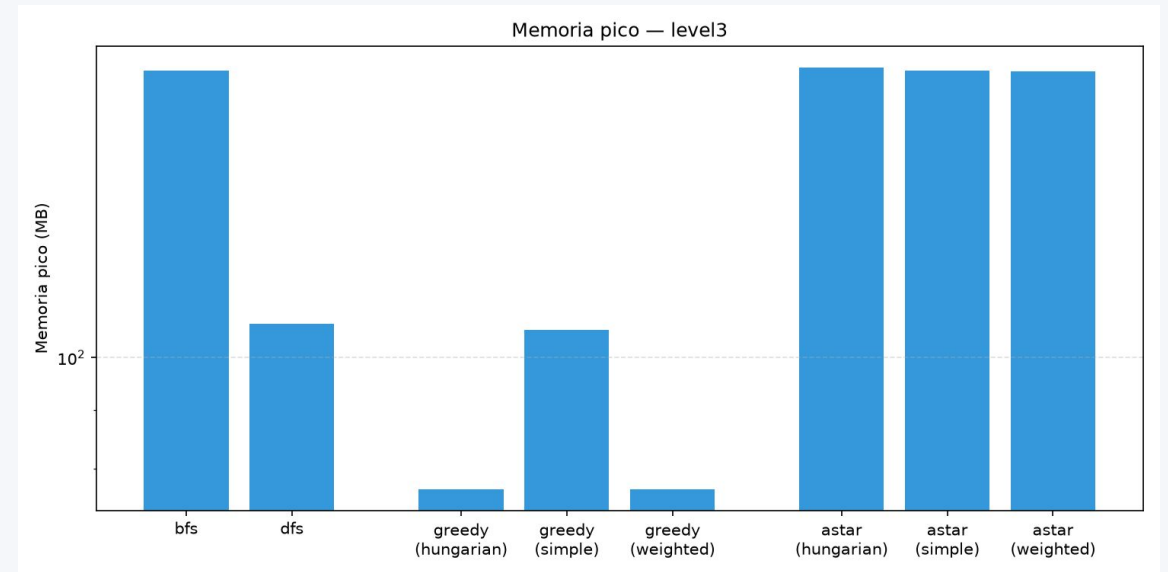
BFS ~24 s / 837k nodos · A simple ~31 s · A* hungarian ~45 s con casi los mismos nodos: es más caro POR NODO, no explora más*

Level 3 — Frontera y memoria

Frontera final vs. máxima

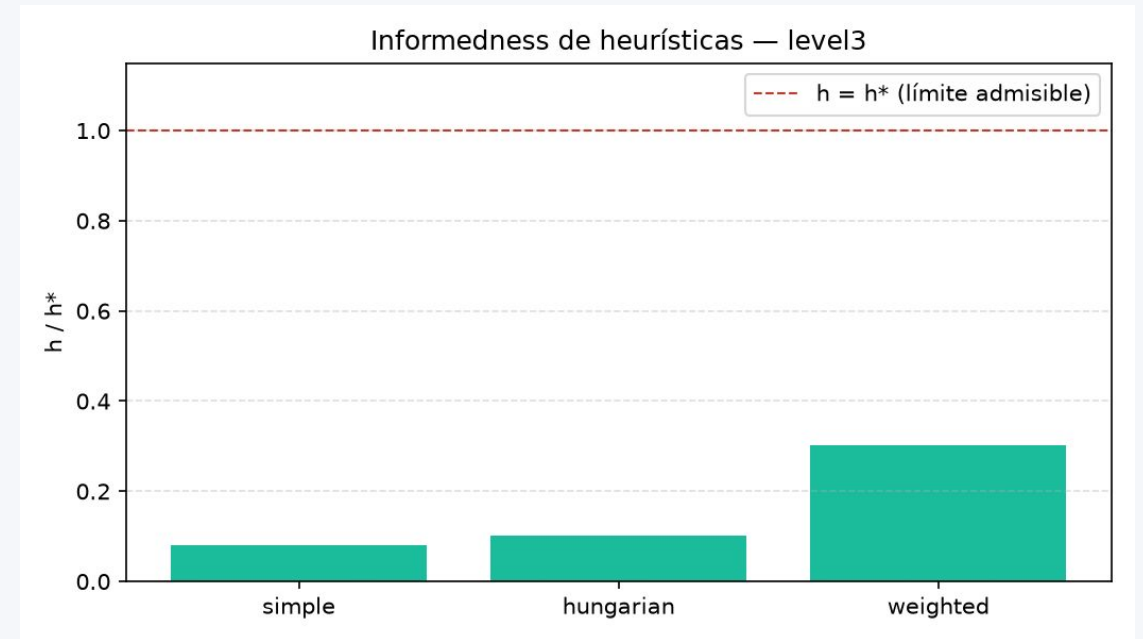
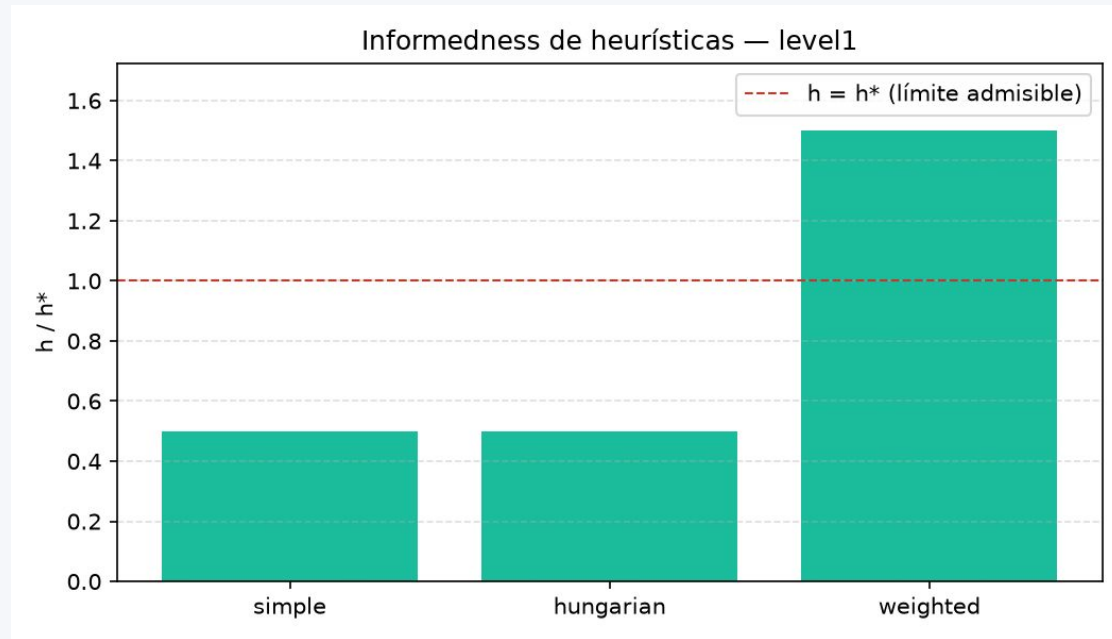


Memoria pico (log, MB)



BFS / A ~178 MB · DFS ~107 MB · Greedy hungarian ~77 MB. La frontera máxima de BFS (7.686) es mucho mayor que la final (37)*

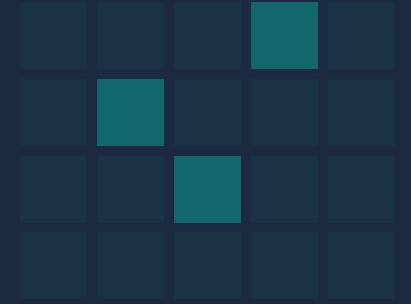
Informedness — ¿qué tan buena es cada heurística?



$h(\text{inicial}) / h^*$ — la línea roja marca $h = h^*$.

Conclusiones

Cinco lecturas de los tres niveles



- 1 BFS y A* con heurística admisible siempre encuentran el costo óptimo — garantía teórica confirmada en los tres niveles
- 2 En el mapa grande, BFS le gana en tiempo a A* admisible (~24 s vs. ~31–45 s) con el mismo costo óptimo de 277
- 3 Greedy y weighted son un trade-off explícito: menos tiempo y memoria a cambio de perder la garantía de optimalidad
- 4 Hungarian es más informada que simple con varias cajas, pero más cara por nodo expandido
- 5 Dead Square Pruning reduce el espacio de búsqueda a la mitad (~2× menos nodos) sin tocar el costo óptimo

Preguntas / Demo GUI

sokoban_game.py

- Teclas A / B / G / D — alternar entre BFS, DFS, Greedy y A* en vivo
- Teclas w / W — alternar heurística hungarian / weighted
- Si quieren el detalle fino de algún número, está en el apéndice

